KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

Student Fee Payment Tracking and Receipt Generation System

Submitted by

AKASH S

25BCS005

Under the Guidance of

DR S BOOPALAN

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

Semester: **II**

DECLARATION

I hereby declare that the project entitled "**Student Fee Payment Tracking and Receipt Generation System"** submitted in partial fulfillment of the requirements for the course **Java programming language** is a **record of my original work** carried out under the guidance of **DR S BOOPALAN**, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

AKASH S

_____

(Your Name)

CERTIFICATE

This is to certify that the project entitled **"Student Fee Payment Tracking and Receipt Generation System"** is a **bona fide work** carried out by **AKASH S BSC CS I A, COMPUTER SCIENCE**, **KG College of Arts & Science**, in partial fulfillment of the requirements for the course **Java programming language**.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place: Coimbatore

Date:10/04/2026

Submitted for the Viva-Voce Examination held on **10/04/2026**

Guide:
(Technical Trainer, KGCAS)

Head of the Department
(Technical Head, KGCAS)

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	
1.1	Objective	
1.2	Overview	
1.3	Features	
2	**Program Analysis**	
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scopes and Limitations	
3	**System Design**	
3.1	Modules	
4	**Methodology**	
4.1	Problem Analysis	
4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	
4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	**Source Code**	
6	**Input and Output Screens**	
7	**Discussion and Conclusion**	
7.1	Discussion	
7.2	Conclusion	
8	**References**	

ABSTRACT

The Digital Document Record Management System is a Java-based application designed to efficiently manage and organize digital documents in a structured manner. In many organizations and institutions, document management is often handled manually or through unorganized digital storage, which can lead to data loss, duplication, and difficulty in retrieval. This project aims to provide a simple and effective solution for storing, managing, and accessing digital records.

The system allows users to add, view, search, update, and delete document records with ease. Each document contains details such as document ID, name, type, date, and file path. The application also provides a search functionality that enables users to quickly locate specific documents based on their name. Additionally, the system supports file handling to store and retrieve data permanently, ensuring data persistence even after the program is closed.

The project is developed using Core Java concepts such as classes, objects, ArrayList, file handling, loops, and conditional statements. It can be implemented as a console-based application, with the possibility of extending it into a graphical user interface using Java Swing or integrating with a database like MySQL for advanced functionality.

The main objective of this system is to improve document management efficiency, reduce manual effort, and provide a reliable way to store and retrieve digital records. The system ensures better organization, faster access, and secure handling of important documents.

This project demonstrates the practical application of object-oriented programming principles in Java and serves as a foundation for developing more advanced document management systems in real-world applications.

1. INTRODUCTION

In the modern digital world, managing documents efficiently is essential for organizations, institutions, and individuals. Traditionally, document records were maintained manually using paper files, which often led to issues such as data loss, misplacement, duplication, and difficulty in retrieval. Even with digital storage, unorganized file management can create confusion and reduce productivity.

With the advancement of computer technology, automated document management systems have become necessary to store, organize, and retrieve documents efficiently. A computerized system helps users maintain records in a structured manner, improves accessibility, and ensures better data security.

The Digital Document Record Management System is a Java-based application designed to manage digital documents effectively. The system allows users to store document details such as document ID, name, type, date, and file path. It provides functionalities to add, view, search, update, and delete document records easily. The system also supports file handling to store data permanently, ensuring that information is not lost after closing the application.

This project is developed using Core Java programming concepts such as classes, objects, methods, loops, conditional statements, and file handling. The application can be implemented as a console-based program and can be further enhanced using Java Swing for a graphical user interface.

The primary goal of this system is to reduce manual effort, improve document organization, and provide faster access to important records. This project also demonstrates how Java can be used to develop practical real-world applications.

1.1 Objective

The main objective of the Digital Document Record Management System is to develop a simple and efficient application using Core Java that automates the process of storing and managing digital documents.

The specific objectives of this project are:

- To design and develop a Java-based document management system.
- To store and manage document details efficiently.
- To provide quick search functionality for retrieving documents.

- To reduce manual errors and improve record management.
- To implement file handling for permanent data storage.
- To demonstrate Core Java concepts such as classes, objects, methods, loops, and conditional statements.
- To provide a user-friendly system for managing digital records.

1.2 Overview

The Digital Document Record Management System is a menu-driven application developed using the Java programming language. It provides a platform where users can manage digital documents in an organized manner.

The system allows users to add new document records by entering details such as name, type, date, and file path. Users can view all stored documents, search for specific documents using their name, update existing records, and delete unwanted documents.

The application uses file handling techniques to store data permanently, ensuring that documents are محفوظ even after the system is closed. This makes the system reliable and efficient for long-term use.

The system simplifies document management by eliminating manual record keeping and reducing the chances of errors. It can be implemented as a console-based application and can be extended into a graphical user interface using Java Swing.

This project demonstrates the practical use of Java programming in developing real-world applications for document storage and management.

1.3 Features

The Digital Document Record Management System includes the following main features:

• **Document Storage** – Allows users to store document details such as name, type, date, and file path.
• **Add Document** – Enables users to add new document records to the system.
• **View Documents** – Displays all stored documents in a structured format.
• **Search Document** – Allows users to search for documents quickly using their name.
• **Update Document** – Enables modification of existing document details.
• **Delete Document** – Allows removal of unwanted or outdated records.
• **File Handling** – Stores data permanently using file handling techniques.
• **User-Friendly Interface** – Provides a simple menu-driven interface for easy operation.
• **Efficient Management** – Improves organization and quick access to documents.

2. PROGRAM ANALYSIS

Program analysis is an important phase in software development where the problem is studied in detail and the requirements of the system are identified. This phase helps in understanding how the system should function and what features are necessary to solve the problem effectively.

The analysis phase focuses on identifying the issues in the existing document management methods and proposing a computerized solution that improves efficiency, organization, and accessibility of records.

2.1 Problem Definition

In many organizations and institutions, document management is often handled manually or through unorganized digital storage systems. This creates several challenges in maintaining and retrieving important records.

Some common problems include:

- Difficulty in locating specific documents
- Risk of data loss or duplication
- Lack of proper organization of files
- Time-consuming manual record handling
- Inefficient document tracking and updating

Manual or poorly managed digital systems increase workload and reduce productivity. It also becomes difficult to maintain large volumes of documents over time.

Therefore, there is a need for an automated system that can efficiently store, manage, and retrieve digital documents in a structured manner.

The Digital Document Record Management System is designed to solve these problems by providing a simple and effective computerized solution for document management.

2.2 Project Overview

The Digital Document Record Management System is a Java-based application developed to simplify the process of managing digital documents.

The system provides a menu-driven interface where users can perform various operations such as adding new documents, viewing existing records, searching for specific documents, updating details, and deleting unwanted records.

Each document contains information such as document ID, name, type, date, and file path. The system automatically manages these records and ensures easy access whenever required.

The system includes the following functionalities:

• Adding new document records
• Viewing all stored documents
• Searching documents by name
• Updating document details
• Deleting unwanted documents
• Storing data permanently using file handling

This project is implemented using Core Java programming concepts and can be developed as a console-based application or extended into a graphical user interface using Java Swing..

2.3 Scopes and Limitations

Scope

The scope of the Digital Document Record Management System includes the following:

• Managing digital document records efficiently
• Storing document details in an organized manner
• Providing quick search and retrieval of documents
• Updating and deleting document records
• Demonstrating the practical use of Java programming concepts

The system can be used in:

• Offices and organizations
• Educational institutions
• Libraries
• Small businesses
• Personal document management

3. SYSTEM DESIGN

System design is the stage where the structure and components of the system are planned before the actual implementation. In this phase, the system is divided into different modules, and the workflow of the application is clearly defined. Proper system design helps in organizing the program efficiently and simplifies development and maintenance.

The Digital Document Record Management System is designed as a menu-driven application where users can manage document records by performing operations such as adding, viewing, searching, updating, and deleting documents.

The system is divided into several modules, each responsible for performing a specific function.

3.1 Modules

Modules

The main modules of the Digital Document Record Management System are explained below:

1. Document Input Module

This module is responsible for adding new document records into the system.

Functions of this module include:

• Accepting user input for document details
• Storing document name, type, date, and file path
• Assigning a unique document ID

Example Input:

Enter Document Name: Report
Enter Type: PDF
Enter Date: 12-03-2026
Enter File Path: C:\Documents\report.pdf

This module ensures that all document details are properly recorded.

2. View Document Module

This module displays all stored document records.

Functions of this module include:

• Retrieving all document records
• Displaying document details in a structured format

Example Output:

------ DOCUMENT LIST ------

ID: 1
Name: Report
Type: PDF
Date: 12-03-2026
Path: C:\Documents\report.pdf

This module helps users view all available documents easily.

3. Search Document Module

This module allows users to search for a specific document using its name.

Functions of this module include:

• Accepting search input from user
• Comparing input with stored records
• Displaying matching document details

Example:

Enter Name to Search: Report

Output: Document Found with details

This module helps in quick retrieval of required documents.

4. Update Document Module

This module allows users to modify existing document details.

Functions of this module include:

• Selecting document using ID
• Updating name, type, date, or file path
• Saving updated data

Example:

Enter ID to Update: 1
Enter New Name: Updated Report

This module ensures that document records remain accurate and up-to-date.

5. Delete Document Module

This module allows users to remove unwanted or outdated documents.

Functions of this module include:

• Selecting document using ID
• Deleting the selected record
• Updating stored data

Example:

Enter ID to Delete: 1

Output: Document Deleted Successfully

6. File Handling Module

This module is responsible for storing and retrieving data permanently.

Functions of this module include:

• Saving document records to a file
• Loading records when the program starts
• Ensuring data persistence

This module ensures that data is not lost after closing the application.

7. Exit Module

This module allows the user to exit the application safely.

When the user selects the exit option, the system terminates the program after saving data.

4. METHODOLOGY

Methodology describes the process followed to develop the project. It explains the steps taken from understanding the problem to designing, implementing, testing, and documenting the system.

For the Digital Document Record Management System, a structured development approach was followed to ensure that the system meets user requirements and functions efficiently.

The development process includes the following stages:

• Problem Analysis
• System Design
• Program Development
• Testing and Validation
• Result and Documentation

4.1. Problem Analysis

In this phase, the problem is studied and analyzed in detail. The requirements of the system are identified based on the need for efficient document management.

The main problems identified in existing systems are:

• Difficulty in managing large numbers of documents
• Risk of data loss or duplication
• Time-consuming document search process
• Lack of proper organization
• No permanent storage in basic manual systems

Based on these problems, the system requirements were defined to develop an automated document management application..

4.2. System Design

In this stage, the overall structure of the system is designed before coding begins.

The system is divided into modules such as:

• Document Input Module
• View Document Module
• Search Document Module
• Update Document Module
• Delete Document Module
• File Handling Module
• Exit Module

The program flow is designed in such a way that users can easily interact with the system through a menu-driven interface.

4.3. Program Development

After designing the system, the program is implemented using the Java programming language.

The following steps were followed during development:

• Creation of Document class and main system class
• Implementation of menu-driven program logic
• Writing code for document operations (add, view, search, update, delete)
• Implementing file handling for data storage
• Handling user input using Scanner class

Core Java concepts such as classes, objects, ArrayList, loops, conditional statements, and file handling were used.

4.4. Testing and Validation

Testing ensures that the program works correctly and produces the expected output.

Different test cases were used to verify system functionality.

Example Test Cases:

Test Case	Expected Result
Add document	Document stored successfully
View documents	All records displayed
Search document	Correct document shown
Update document	Data updated successfully
Delete document	Record removed correctly
Invalid input	Error handled properly

This phase ensures that the program runs without errors and produces accurate results.

4.5. Result and Documentation

After successful testing, the results of the system are documented.

The system allows users to:

• Add new documents
• View stored documents
• Search and retrieve documents
• Update and delete records

The results are demonstrated through program execution and output screens.

Proper documentation is prepared to explain the design, implementation, and working of the system.

4.6. Algorithm

1. Start the program.
2. Display the menu options (Add, View, Search, Update, Delete, Exit).
3. Ask the user to select an option.
4. If the user selects Add
5. If the user selects View
6. If the user selects Search
7. If the user selects Update
8. If the user selects Delete
9. Save the data to file.
10. End the program.

4.7. Flowchart

Students should draw a flowchart showing program flow.

Example flow:

- Start
  ↓
  Display Menu
  ↓
  Select Option
  ↓
  Perform Operation
  ↓
  Display Result
  ↓
  Exit?
  ↓
  End
- Students can draw using:
- • draw.io
  • Lucidchart
  • MS Visio

5. SOURCE CODE

Include important parts of the Java code such as:

Main class

```
import java.util.*;


class DocumentSystem {
   static ArrayList<Document> docs = new ArrayList<>();
   static Scanner sc = new Scanner(System.in);


   public static void main(String[] args) {


      while (true) {

System.out.println("\n1.Add\n2.View\n3.Search\n4.Update\n5.Delete\n6.Exit");
         int choice = sc.nextInt();


         switch (choice) {
            case 1: addDocument(); break;
            case 2: viewDocuments(); break;
            case 3: searchDocument(); break;
            case 4: updateDocument(); break;
            case 5: deleteDocument(); break;
            case 6: System.exit(0);
            default: System.out.println("Invalid choice");
         }
      }
   }
```

```
}
```

Menu logic

```
static void addDocument() {
    sc.nextLine();
    System.out.print("Enter Name: ");
    String name = sc.nextLine();

    System.out.print("Enter Type: ");
    String type = sc.nextLine();

    System.out.print("Enter Date: ");
    String date = sc.nextLine();

    System.out.print("Enter Path: ");
    String path = sc.nextLine();

    docs.add(new Document(docs.size()+1, name, type, date, path));
    System.out.println("Document Added!");
}
```

Search Logic

```
static void searchDocument() {
    sc.nextLine();
    System.out.print("Enter name: ");
    String search = sc.nextLine();

    for (Document d : docs) {
        if (d.name.equalsIgnoreCase(search)) {
            d.display();
```

```
        }
    }
}
```

Update Logic

```
static void updateDocument() {
    System.out.print("Enter ID: ");
    int id = sc.nextInt();
    sc.nextLine();

    for (Document d : docs) {
        if (d.id == id) {
            System.out.print("New Name: ");
            d.name = sc.nextLine();
            System.out.println("Updated!");
        }
    }
}
```

Delete Logic

```
static void deleteDocument() {
    System.out.print("Enter ID: ");
    int id = sc.nextInt();

    docs.removeIf(d -> d.id == id);
    System.out.println("Deleted!");
}
```

Example Logic

```
docs.add(new Document(id, name, type, date, path));
```

```
if (d.name.equalsIgnoreCase(search))
```

```
docs.removeIf(d -> d.id == id);
```

6. INPUT AND OUTPUT SCREENS

Include screenshots of:

- Add Document Input Screen

```
1. Add Student
2. Pay Fee
3. View Student
4. Generate Receipt
5. Exit
Enter choice: 1
Enter ID: 101
Enter Name: Ameer
Enter Total Fee: 5000
Student added successfully!
```

- Document Added Confirmation

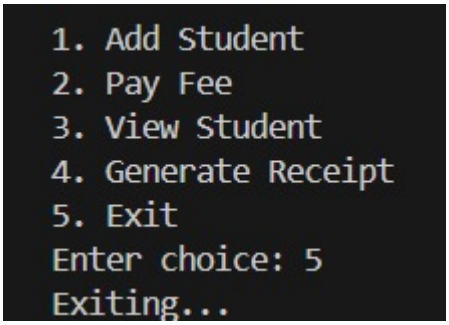
```
1. Add Student
2. Pay Fee
3. View Student
4. Generate Receipt
5. Exit
Enter choice: 2
Enter Student ID: 101
Enter Amount: 5000
Payment successful!
```

```
1. Add Student
2. Pay Fee
3. View Student
4. Generate Receipt
5. Exit
Enter choice: 3
Enter Student ID: 101
ID: 101
Name: Ameer
Total Fee: 5000.0
Paid Fee: 5000.0
Balance: 0.0
```

```
1. Add Student
2. Pay Fee
3. View Student
4. Generate Receipt
5. Exit
Enter choice: 4
Enter Student ID: 101
Enter Amount Paid: 5000

===== RECEIPT =====
Student ID: 101
Name: Ameer
Amount Paid: 5000.0
Total Paid: 5000.0
Balance: 0.0
==================
```

Exit Screen



Each screenshot should be labeled.

Example:

Figure 1: System Start Screen

Figure 2: Main Menu Screen

Figure 3: Add Document Screen

Figure 4: Document Added Output

Figure 5: View Documents Output

Figure 6: Search Result Screen

Figure 7: Update Document Screen

Figure 8: Delete Confirmation Screen

Figure 9: Data Saved Message

Figure 10: Exit Screen

7. DISCUSSION & CONCLUSION

7.1 Discussion

The Digital Document Record Management System was developed to automate the process of managing digital documents efficiently. The system allows users to add, view, search, update, and delete document records easily.

During the development process, Core Java programming concepts such as classes, objects, loops, conditional statements, methods, and file handling were applied. The system successfully manages document records and ensures proper storage and retrieval of data.

The system was tested with different inputs to verify that all operations such as adding, searching, updating, and deleting documents work correctly. The results show that the system reduces manual effort and improves document management efficiency.

The use of file handling ensures that the data is stored permanently, making the system reliable for continuous usage.

7.2 Conclusion

The Digital Document Record Management System successfully demonstrates the practical application of Core Java programming concepts in developing a real-world application.

The system simplifies document management by organizing records in a structured manner and providing quick access to information. It reduces manual work, minimizes errors, and improves efficiency in handling digital documents.

This project helps students understand how Java programming can be used to build useful software systems. The project can be further enhanced by adding advanced features such as database connectivity, graphical user interfaces, user authentication, and cloud storage integration.

8. REFERENCES

The following resources were used during the development of this project:

Books

• Herbert Schildt – *Java: The Complete Reference*

Websites

• https://www.oracle.com/java
• https://www.geeksforgeeks.org
• https://www.w3schools.com
• https://www.javatpoint.com